Automated Tabular Data Cleansing & Executive Report Generator

Author

Anurag Kumar Poddar

Data Analyst | Python | SQL | Power BI | Advanced Excel | Data Visualization

◈ GitHub Profile- https://github.com/anuragkumarpoddar-ui ◈ LinkedIn Profile- https://www.linkedin.com/in/anurag-kumar-poddar-51239596/

Project Overview

The Automated Tabular Data Cleansing & Executive Report Generator (CorporateReportBuilder) is a Python-based desktop application designed to streamline ad-hoc data analysis and executive reporting[cite: 1].

In corporate environments, raw datasets frequently suffer from formatting inconsistencies—such as numeric values stored as strings with comma separators, unstandardized categorical text, and missing values[cite: 1]. This tool automates the ingestion, sanitization, aggregation, and visualization of raw tabular files (.csv, .xlsx, .xls), enabling users to generate clean summary tables and publication-ready charts through an intuitive graphical interface without writing a single line of code

In [ ]:
```
import os
import tkinter as tk
from tkinter import filedialog, messagebox, ttk

import pandas as pd

import matplotlib
matplotlib.use("TkAgg")
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg


class CorporateReportBuilder:
    def __init__(self, root):
        self.root = root
        self.root.title("Corporate Data Analyzer (Report + Chart + Export
```

```
        self.root.geometry("1100x700")
        self.root.resizable(True, True)

        self.file_path = ""
        self.df = None
        self.report_df = None
        self.canvas = None
        self.current_figure = None

        self.agg_map = {
            "Sum": "sum",
            "Mean": "mean",
            "Average": "mean",
            "Max": "max",
            "Min": "min",
            "Count": "count",
            "Median": "median",
        }

        self._build_ui()

    # ---------------- UI ----------------
    def _build_ui(self):
        header = tk.Label(self.root, text="Corporate Data Analyzer", font
        header.pack(pady=10)

        # File row
        file_frame = tk.Frame(self.root)
        file_frame.pack(fill="x", padx=15, pady=5)

        tk.Label(file_frame, text="Select CSV/Excel:", font=("Arial", 10)

        tk.Button(file_frame, text="Browse", command=self.browse_file, wi
        tk.Button(file_frame, text="Read", command=self.read_file, width=

        self.file_lbl = tk.Label(file_frame, text="No file selected", fg=
        self.file_lbl.pack(side="left", padx=10)

        # Info row
        info_frame = tk.LabelFrame(self.root, text="File Info", padx=10,
        info_frame.pack(fill="x", padx=15, pady=8)

        self.info_text = tk.Text(info_frame, height=4)
        self.info_text.pack(fill="x")

        # Controls (dropdowns)
        controls = tk.LabelFrame(self.root, text="Build Report (GroupBy +
        controls.pack(fill="x", padx=15, pady=8)

        row1 = tk.Frame(controls)
        row1.pack(fill="x", pady=3)

        tk.Label(row1, text="Group By (Text column):").pack(side="left")
        self.group_col_var = tk.StringVar()
```

```
        self.group_col_cb = ttk.Combobox(row1, textvariable=self.group_co
        self.group_col_cb.pack(side="left", padx=8)

        tk.Label(row1, text="Aggregation:").pack(side="left", padx=(10, 0
        self.agg_var = tk.StringVar()
        self.agg_cb = ttk.Combobox(
            row1,
            textvariable=self.agg_var,
            state="disabled",
            values=list(self.agg_map.keys()),
            width=18,
        )
        self.agg_cb.pack(side="left", padx=8)

        tk.Label(row1, text="Value (Numeric column):").pack(side="left",
        self.value_col_var = tk.StringVar()
        self.value_col_cb = ttk.Combobox(row1, textvariable=self.value_co
        self.value_col_cb.pack(side="left", padx=8)

        row2 = tk.Frame(controls)
        row2.pack(fill="x", pady=8)

        tk.Button(row2, text="Preview Report", command=self.preview_repor
            side="left", padx=5
        )

        tk.Label(row2, text="Export as:").pack(side="left", padx=(15, 0))
        self.export_format_var = tk.StringVar(value="Excel (.xlsx)")
        self.export_format_cb = ttk.Combobox(
            row2,
            textvariable=self.export_format_var,
            state="readonly",
            values=["Excel (.xlsx)", "CSV (.csv)"],
            width=15,
        )
        self.export_format_cb.pack(side="left", padx=8)

        tk.Button(row2, text="Export Report", command=self.export_report,

        # Chart controls
        chart_frame = tk.LabelFrame(self.root, text="Chart Builder", padx
        chart_frame.pack(fill="x", padx=15, pady=8)

        c_row = tk.Frame(chart_frame)
        c_row.pack(fill="x", pady=5)

        tk.Label(c_row, text="Chart Type:").pack(side="left")
        self.chart_type_var = tk.StringVar(value="Bar")
        self.chart_type_cb = ttk.Combobox(
            c_row,
            textvariable=self.chart_type_var,
            state="readonly",
            values=["Bar", "Column", "Pie", "Line"],
            width=12,
```

```
        )
        self.chart_type_cb.pack(side="left", padx=8)

        tk.Button(c_row, text="Preview Chart", command=self.preview_chart
            side="left", padx=5
        )
        tk.Button(c_row, text="Export Chart (PNG)", command=self.export_c

        # Main area: table + chart
        main = tk.Frame(self.root)
        main.pack(fill="both", expand=True, padx=15, pady=10)

        # Table
        table_box = tk.LabelFrame(main, text="Report Preview", padx=8, pa
        table_box.pack(side="left", fill="both", expand=True)

        self.tree = ttk.Treeview(table_box, columns=("A", "B"), show="hea
        self.tree.heading("A", text="Group")
        self.tree.heading("B", text="Value")
        self.tree.column("A", width=220, anchor="w")
        self.tree.column("B", width=140, anchor="e")

        yscroll = ttk.Scrollbar(table_box, orient="vertical", command=sel
        self.tree.configure(yscrollcommand=yscroll.set)

        self.tree.pack(side="left", fill="both", expand=True)
        yscroll.pack(side="right", fill="y")

        # Chart
        chart_box = tk.LabelFrame(main, text="Chart Preview", padx=8, pad
        chart_box.pack(side="right", fill="both", expand=True)

        self.chart_container = tk.Frame(chart_box)
        self.chart_container.pack(fill="both", expand=True)

    # ---------------- Helpers ----------------
    def _set_info(self, text: str):
        self.info_text.delete("1.0", tk.END)
        self.info_text.insert(tk.END, text)

    def _input_folder(self) -> str:
        if not self.file_path:
            return ""
        return os.path.dirname(os.path.abspath(self.file_path))

    def _safe_numeric_convert(self, series: pd.Series) -> pd.Series:
        # Convert strings like "12,000" safely to numeric; non-convertibl
        return pd.to_numeric(series.astype(str).str.replace(",", "", rege

    # ---------------- Actions ----------------
    def browse_file(self):
        path = filedialog.askopenfilename(
            title="Select a CSV or Excel file",
            filetypes=[("CSV Files", "*.csv"), ("Excel Files", "*.xlsx *.
```

```
        )
        if path:
            self.file_path = path
            self.file_lbl.config(text=os.path.basename(path))

    def read_file(self):
        if not self.file_path:
            messagebox.showerror("Error", "Please select a file first.")
            return

        try:
            if self.file_path.lower().endswith(".csv"):
                self.df = pd.read_csv(self.file_path)
            else:
                self.df = pd.read_excel(self.file_path)

            # Basic info
            rows, cols = self.df.shape
            headings = list(self.df.columns)

            info = (
                f"Rows: {rows}\n"
                f"Columns: {cols}\n"
                f"Column Headings:\n- " + "\n- ".join(map(str, headings))
            )
            self._set_info(info)

            # Identify text vs numeric columns
            text_cols = list(self.df.select_dtypes(include=["object"]).co

            # For numeric columns, also consider columns that are numeric
            numeric_cols = list(self.df.select_dtypes(include=["number"])
            for c in self.df.columns:
                if c in numeric_cols:
                    continue
                # try convert sample
                converted = self._safe_numeric_convert(self.df[c])
                if converted.notna().sum() >= max(3, int(0.6 * len(self.d
                    numeric_cols.append(c)

            # Update dropdowns
            self.group_col_cb.config(state="readonly", values=text_cols)
            self.value_col_cb.config(state="readonly", values=numeric_col
            self.agg_cb.config(state="readonly")

            # default selections
            if text_cols:
                self.group_col_var.set(text_cols[0])
            else:
                self.group_col_var.set("")

            if numeric_cols:
                self.value_col_var.set(numeric_cols[0])
            else:
```

```
                self.value_col_var.set("")

            self.agg_var.set("Sum")

            messagebox.showinfo("Success", "File read successfully. Now s

        except Exception as e:
            messagebox.showerror("Read Error", f"Could not read file.\n\n

    def preview_report(self):
        if self.df is None:
            messagebox.showerror("Error", "Please Read the file first.")
            return

        group_col = self.group_col_var.get().strip()
        agg_label = self.agg_var.get().strip()
        value_col = self.value_col_var.get().strip()

        if not group_col or not agg_label or not value_col:
            messagebox.showerror("Error", "Please select Group By column,
            return

        try:
            agg_func = self.agg_map.get(agg_label, "sum")
            df_work = self.df.copy()

            # Clean group column
            df_work[group_col] = df_work[group_col].astype(str).str.strip

            # Ensure numeric
            df_work[value_col] = self._safe_numeric_convert(df_work[value
            df_work = df_work.dropna(subset=[value_col])

            # GroupBy + Aggregation
            report = (
                df_work.groupby(group_col)[value_col]
                .agg(agg_func)
                .reset_index()
                .rename(columns={group_col: "Group", value_col: "Value"})
                .sort_values("Value", ascending=False)
            )

            self.report_df = report

            # Show in table (Treeview)
            self._render_table(report)

        except Exception as e:
            messagebox.showerror("Report Error", f"Could not build report

    def _render_table(self, report: pd.DataFrame):
        # Clear existing
        for item in self.tree.get_children():
            self.tree.delete(item)
```

```
        # Update headings
        self.tree.heading("A", text=report.columns[0])
        self.tree.heading("B", text=report.columns[1])

        # Insert rows
        for _, r in report.iterrows():
            a = str(r.iloc[0])
            b = r.iloc[1]
            try:
                b_disp = f"{float(b):,.2f}"
            except Exception:
                b_disp = str(b)
            self.tree.insert("", "end", values=(a, b_disp))

    def export_report(self):
        if self.report_df is None or self.report_df.empty:
            messagebox.showerror("Error", "No report to export. Click 'Pr
            return

        folder = self._input_folder()
        if not folder:
            messagebox.showerror("Error", "Input folder not found.")
            return

        fmt = self.export_format_var.get()
        base_name = "report_output"

        try:
            if fmt.startswith("Excel"):
                out_path = os.path.join(folder, base_name + ".xlsx")
                self.report_df.to_excel(out_path, index=False)
            else:
                out_path = os.path.join(folder, base_name + ".csv")
                self.report_df.to_csv(out_path, index=False)

            messagebox.showinfo("Exported", f"Report exported successfull
        except Exception as e:
            messagebox.showerror("Export Error", f"Could not export repor

    def preview_chart(self):
        if self.report_df is None or self.report_df.empty:
            messagebox.showerror("Error", "No report to chart. Click 'Pre
            return

        chart_type = self.chart_type_var.get()
        data = self.report_df.copy()

        # Limit to top N for readability
        top_n = 10 if len(data) > 10 else len(data)
        data = data.head(top_n)

        try:
            # Clear old canvas
```

```
            for widget in self.chart_container.winfo_children():
                widget.destroy()

            fig = Figure(figsize=(5.2, 4.2), dpi=100)
            ax = fig.add_subplot(111)

            labels = data.iloc[:, 0].astype(str).tolist()
            values = data.iloc[:, 1].astype(float).tolist()

            if chart_type in ["Bar", "Column"]:
                if chart_type == "Bar":
                    ax.barh(labels[::-1], values[::-1])
                    ax.set_xlabel("Value")
                    ax.set_ylabel(data.columns[0])
                else:
                    ax.bar(labels, values)
                    ax.set_ylabel("Value")
                    ax.tick_params(axis="x", rotation=45)

            elif chart_type == "Line":
                ax.plot(labels, values, marker="o")
                ax.set_ylabel("Value")
                ax.tick_params(axis="x", rotation=45)

            elif chart_type == "Pie":
                # Pie works best when labels are few
                pie_n = 6 if len(values) > 6 else len(values)
                ax.pie(values[:pie_n], labels=labels[:pie_n], autopct="%1
                ax.axis("equal")

            ax.set_title(f"{chart_type} Chart: {data.columns[1]} by {data
            fig.tight_layout()

            self.current_figure = fig
            self.canvas = FigureCanvasTkAgg(fig, master=self.chart_contai
            self.canvas.draw()
            self.canvas.get_tk_widget().pack(fill="both", expand=True)

        except Exception as e:
            messagebox.showerror("Chart Error", f"Could not preview chart

    def export_chart(self):
        if self.current_figure is None:
            messagebox.showerror("Error", "No chart to export. Click 'Pre
            return

        folder = self._input_folder()
        if not folder:
            messagebox.showerror("Error", "Input folder not found.")
            return

        out_path = os.path.join(folder, "chart_output.png")
        try:
            self.current_figure.savefig(out_path, dpi=200)
```

```
            messagebox.showinfo("Exported", f"Chart exported successfully
        except Exception as e:
            messagebox.showerror("Export Error", f"Could not export chart


if __name__ == "__main__":
    root = tk.Tk()
    app = CorporateReportBuilder(root)
    root.mainloop()
```

In [ ]:

In [ ]:

In [ ]: